

HANGMAN GAME USING PYTHON

Submitted by:

Name: Saikam Bhavya Sri

College: Vigna's Nirula Institute of Technology and Science for Women

Department: Electronics and Communication Engineering

Aim:

To develop a simple text-based Hangman game in Python where the player guesses a hidden word one letter at a time, with a maximum of 6 incorrect guesses.

Software Requirements:

Details	Requirement
	Language
Python	
	IDE
VS Code/IDLE/PyCharm	
	Modules Used
Random	

Platform

Windows/Linux

Algorithm:

1. Import random module.
2. Create a list of predefined words.
3. Select one random word.
4. Display blanks for the hidden word.
5. Ask the user for a letter.
6. If the letter exists, reveal it.
7. Otherwise, reduce chances.
8. Repeat until:
9. word guessed OR
10. chances become 0.
11. Display result.

Program Code:

```
import random

# List of predefined words
words = ["python", "apple", "tiger", "planet", "guitar"]

# Choose a random word
word = random.choice(words)

# Variables
guessed_letters = []
wrong_guesses = 0
```

```
max_wrong_guesses = 6
```

```
# Display hidden word
```

```
hidden_word = ["_"] * len(word)
```

```
print("🎮 Welcome to Hangman!")
```

```
print("Guess the word one letter at a time.")
```

```
# Game loop
```

```
while wrong_guesses < max_wrong_guesses and "_" in hidden_word:
```

```
    print("\nWord:", " ".join(hidden_word))
```

```
    print("Wrong guesses left:", max_wrong_guesses - wrong_guesses)
```

```
    guess = input("Enter a letter: ").lower()
```

```
# Check if already guessed
```

```
if guess in guessed_letters:
```

```
    print("You already guessed that letter.")
```

```
    continue
```

```
guessed_letters.append(guess)
```

```
# Check if guess is correct
```

```
if guess in word:
```

```
    print("Correct!")
```

```
# Reveal matching letters
```

```
for i in range(len(word)):
```

```
    if word[i] == guess:
```

```

        hidden_word[i] = guess

    else:

        wrong_guesses += 1

        print("Wrong guess!")

# Game result

if "_" not in hidden_word:

    print("\n🎉 Congratulations! You guessed the word:", word)

else:

    print("\n❌ Game Over! The word was:", word)

```

Output:

```

main.py
26
27 # Check if already guessed
28 if guess in guessed_letters:
29     print("You already guessed that letter.")
30     continue
31
32 guessed_letters.append(guess)
33
34 # Check if guess is correct
35 if guess in word:
36     print("Correct!")
37
38 # Reveal matching letters
39 for i in range(len(word)):
40     if word[i] == guess:
41         hidden_word[i] = guess
42 else:
43     wrong_guesses += 1
44     print("Wrong guess!")
45
46 # Game result
47 if "_" not in hidden_word:
48     print("\n🎉 Congratulations! You guessed the word:", word)
49 else:
50     print("\n❌ Game Over! The word was:", word)

```

Output

```

Welcome to Hangman!
Guess the word one letter at a time.

Word: _ _ _ _ _
Wrong guesses left: 6
Enter a letter: b
Wrong guess!

Word: _ _ _ _ _
Wrong guesses left: 5
Enter a letter: r
Correct!

Word: _ _ _ _ r
Wrong guesses left: 5
Enter a letter: h
Wrong guess!

Word: _ _ _ _ r
Wrong guesses left: 4
Enter a letter: a
Wrong guess!

Word: _ _ _ _ r
Wrong guesses left: 3

```

The screenshot shows a web browser window with the URL `programiz.com/python-programming/online-compiler/`. The page title is "Programiz Python Online Compiler". The interface includes a "main.py" tab, a "Run" button, and an "Output" panel. The code in the editor is a Python script for a word guessing game. The output panel shows the execution results, including the word being guessed, the number of wrong guesses left, and the final game result.

```
26
27 # Check if already guessed
28 if guess in guessed_letters:
29     print("You already guessed that letter.")
30     continue
31
32 guessed_letters.append(guess)
33
34 # Check if guess is correct
35 if guess in word:
36     print("Correct!")
37
38 # Reveal matching letters
39 for i in range(len(word)):
40     if word[i] == guess:
41         hidden_word[i] = guess
42 else:
43     wrong_guesses += 1
44     print("Wrong guess!")
45
46 # Game result
47 if "_" not in hidden_word:
48     print("\n🎉 Congratulations! You guessed the word:", word)
49 else:
50     print("\n❌ Game Over! The word was:", word)
```

Output:

```
Wrong guess!
Word: _ _ _ _ r
Wrong guesses left: 4
Enter a letter: a
Wrong guess!

Word: _ _ _ _ r
Wrong guesses left: 3
Enter a letter: l
Wrong guess!

Word: _ _ _ _ r
Wrong guesses left: 2
Enter a letter: p
Wrong guess!

Word: _ _ _ _ r
Wrong guesses left: 1
Enter a letter: s
Wrong guess!

❌ Game Over! The word was: tiger

=== Code Execution Successful ===
```

Output Screenshots of the code while simulation.

Explanation/Working

- The program randomly chooses one word.
- Player enters one letter.
- Correct letters are shown.
- Wrong letters reduce attempts.
- Maximum 6 wrong guesses allowed.
- Game ends with a win or a loss.

Features Included:

- Random word selection
- 5 predefined words
- Maximum 6 incorrect guesses
- Tracks guessed letters
- Displays progress after each turn
- Win/Lose messages.

Result:

Successfully developed and executed a text-based Hangman game using Python with random word selection and user interaction.

Conclusion:

The Hangman game was successfully developed using Python. This project helped me understand basic Python concepts, including lists, strings, conditional statements, loops, and the random module. The program allows the user to guess letters one at a time with a limit of 6 incorrect attempts. The game executes correctly and provides an interactive console-based experience. Through this task, practical knowledge of Python programming and problem-solving was improved.